

Software Requirements Specification (SRS)

Unifying Business Requirements (BRD) & Functional Requirements (FRS)

Project: HireSense (AI-Powered Applicant Tracking System)

Document Version: 1.0.0

Author: Antigravity Systems Analyst

Status: Approved

Target Platform: Web & Mobile Dashboard Interfaces

PART I: BUSINESS REQUIREMENTS DOCUMENT (BRD)

1. Business Context & Strategic Objectives

1.1 Project Background

In the modern corporate recruitment environment, human resource departments face massive bottlenecks when screening candidates for entry-level positions and student internship roles. Recruiters are overwhelmed with hundreds of poorly formatted resumes per job post, and traditional Applicant Tracking Systems (ATS) rely on basic keyword indexing, leading to high keyword-stuffing exploits and low talent retention.

HireSense aims to solve this operational friction by deploying semantic dense-vector similarity matching, heuristic taxonomy parsing, and low-latency asynchronous processing.

1.2 Strategic Objectives & KPIs

The success of the HireSense implementation is measured against key operational performance indicators:

| Strategic Objective | Metric / Benchmark | Target Improvement |

| :--- | :--- | :--- |

| Recruiter Screening Speed | Average hours spent screening 500 candidate resumes | Decrease from 12 hours to < 30 minutes |

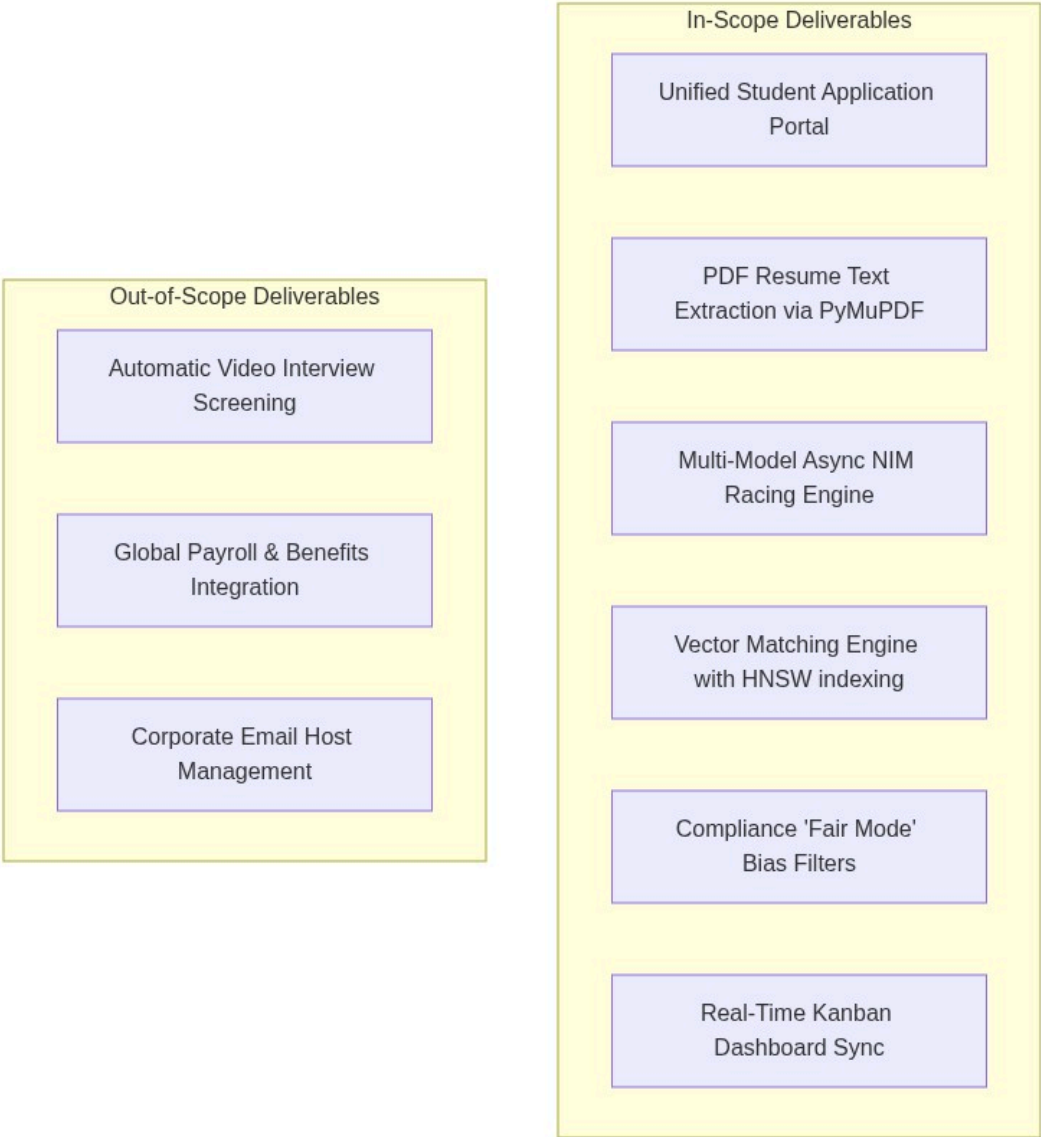
| Time-to-Hire (TTH) | Days from job post creation to signed candidate offer | Decrease from 45 days to 14 days |

| Parsing Precision | Accuracy of extracted entity fields compared to manual audit | Reach > 98% accuracy on standard PDFs |

| Explainability Rate | Percentage of AI scoring matches with full breakdown | Reach 100% auditability (0% "black box" queries) |

| Candidate Drop-off Rate | Ratio of applicants abandoning applications due to lag | Reduce from 35% to < 2% |

1.3 Scope of Project

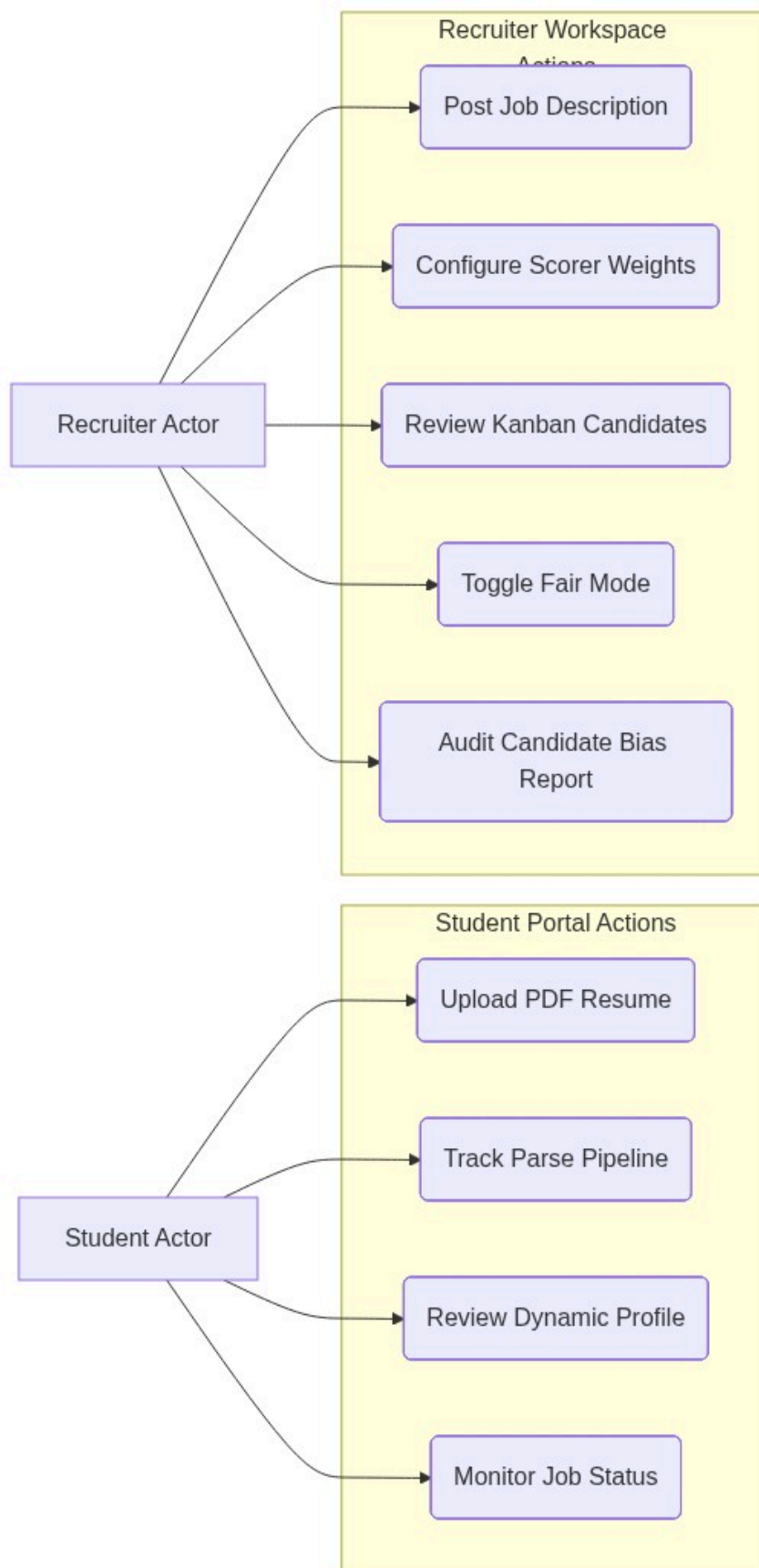


1.4 Business Rules & Compliance

- 1. NYC Local Law 144 Compliance: Any automated employment decision tool (AEDT) must provide full explainability. Black-box evaluations are forbidden. The matching algorithm must document scoring metrics (Semantic, Skill, Experience) and store audits for regulatory evaluation.
- 2. "Fair Mode" Mandate: The platform must support a bias-reduction toggle. When active, chronological factors (Years of Experience) are set to 0% weight, dynamically shifting emphasis to skill compatibility and semantic intent.
- 3. Strict Data Scoping: Candidates must have absolute privacy. Recruiters can only access resumes submitted to their specific job description channels. Applicants must have visibility of their active parsing pipelines without accessing background recruiter evaluations.

PART II: FUNCTIONAL REQUIREMENTS SPECIFICATION (FRS)

2. User Persona Journeys & Use Cases



2.1 Candidate Journey: Alex (The Student Applicant)

1. Trigger: Alex enters the HireSense Student Portal and drags a PDF resume into the drop zone.

2. Functional Actions:

- System validates size limit (< 10MB) and format (PDF/DOCX).
- System initiates background extraction task, displaying a real-time cinematic progress spinner.
- System auto-fills the candidate's profile form using parsed details (Name, Skills, Work History).
- Alex confirms the auto-populated details and clicks "Submit Application."

3. Post-Condition: Alex is redirected to a personalized dashboard streaming real-time status updates (Pending Screened Interviewing).

2.2 Recruiter Journey: Sarah (The High-Volume Recruiter)

1. Trigger: Sarah logs in to post a new Software Engineer Job Description (JD).

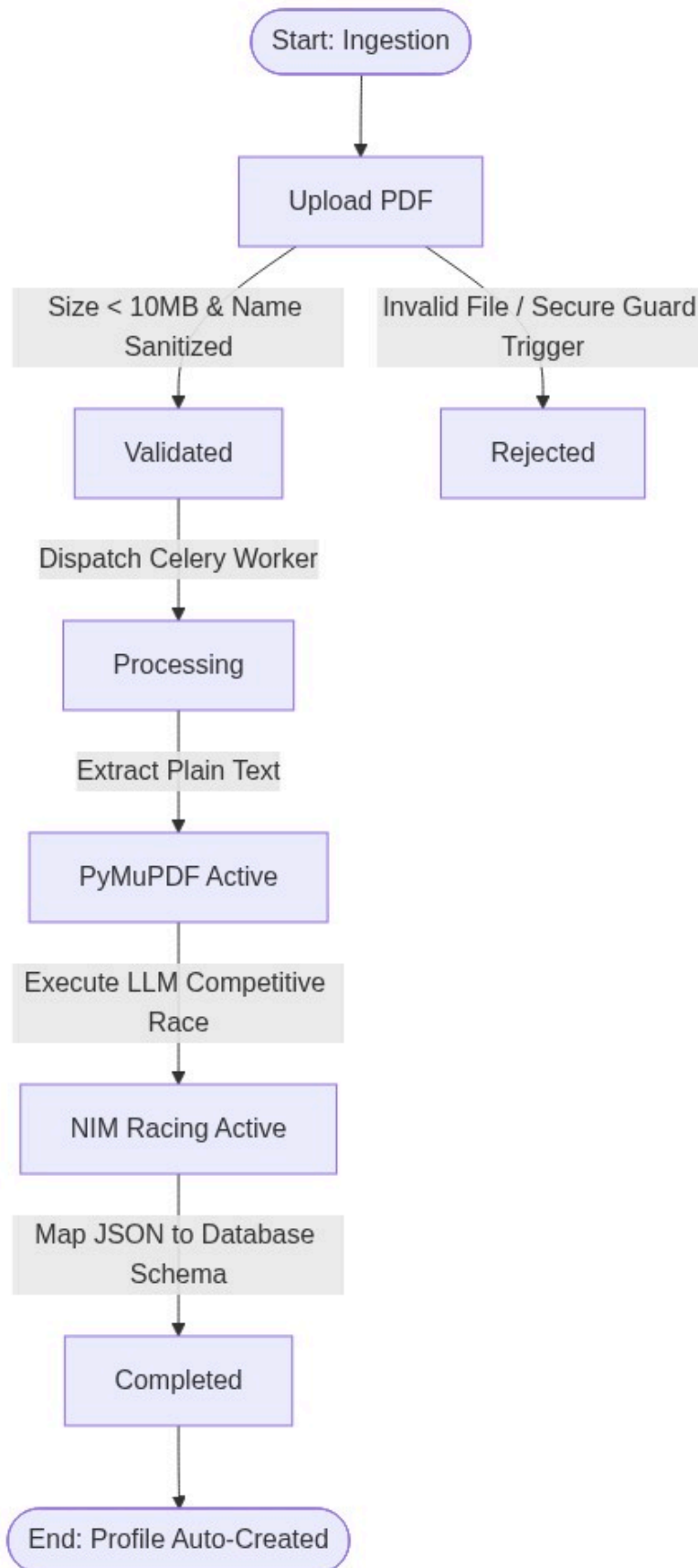
2. Functional Actions:

- Sarah types the JD requirements and skill weights.
 - System computes and indexes the dense vector representation of the JD in < 500ms.
 - Sarah activates "Fair Mode" to ensure compliance with bias regulations for early-career students.
 - Sarah monitors the dynamic Kanban Board, dragging applicants across status columns with instant WebSocket synchronization.
-

3. Detailed Functional Modules Specification

3.1 Module 1: PDF Resume Ingestion & Parsing Services

- Description: Ingests, sanitizes, extracts, and parses candidate details from resume documents.



- System Processing Steps:

1. The API checks the file extension and header signature.

- 2. The script runs `secure_filename` to sanitize path injections.
- 3. An asynchronous task `process_resume` is enqueued in Celery.
- 4. Plain text is extracted page-by-page using PyMuPDF.
- 5. The extraction triggers parallel API racing across meta/llama endpoints to parse entities.
- Input Data: Uploaded PDF file stream.
- Output Data: Structured JSON document conforming to the Pydantic schema below:

```
{
  "candidate_name": "Alex Mercer",
  "contact": {
    "email": "alex.mercer@gmail.com",
    "phone": "+1-555-0199"
  },
  "skills": ["Python", "FastAPI", "React", "Docker", "PostgreSQL"],
  "experience_years": 2,
  "education": [
    {
      "institution": "State University",
      "degree": "B.S. in Computer Science"
    }
  ]
}
```

- Error Handling: If the PDF is corrupted, the system updates the state to Failed and notifies the client via a WebSocket alert, prompting a fresh upload.

3.2 Module 2: Job Description Embedding & HNSW Indexing

- Description: Computes semantic dense vectors for newly created job descriptions to facilitate ultra-fast nearest-neighbor searches.
- Processing Rules:
- Embeddings must have exactly 384 dimensions (all-MiniLM-L6-v2).
- Database writes must commit both the raw text and the float array vector.
- HNSW index parameters must target `m = 16` and `ef_construction = 64`.
- Response Benchmark: Embedding calculation and indexing must commit within < 300ms.

3.3 Module 3: Multidimensional Scoring Engine

- Description: Computes overall compatibility matching scores (0% - 100%) between candidate profiles and job requirements.
- Weighting Matrices:

Dimension	Standard Mode Weight	Fair Mode Weight	Calculation Methodology
:---	:---	:---	:---
Semantic Similarity	40%	55%	Cosine similarity between JD and Resume vectors (pgvector <=> operator)
Skill Compatibility	30%	35%	Keyword overlap intersection ratio over job required skillsets
Experience Fit	20%	0%	Heuristic absolute difference comparison with target threshold

PART III: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

4. System & Non-Functional Requirements

4.1 Performance & Sub-Second Latency

- Inference Speeds: The NIM Racing Engine must resolve concurrent LLM queries within < 2.5s under normal workloads, falling back to local heuristic engines if latency exceeds 8.0s.
- Database Search: Semantic vector searches utilizing HNSW indexing must execute within < 100ms for datasets up to 50,000 resume records.
- State Broadcasting: WebSocket updates of Kanban transitions must distribute to all subscribed clients in < 200ms.

4.2 Security & Data Hardening

- logical Isolation (RLS): Row-Level Security policies must explicitly evaluate `auth.uid()::text = user_id` for every query, preventing cross-tenant leaks.
- Upload Boundaries: payload limits must be enforced at the gateway level to instantly reject streams larger than 10MB.
- Input Sanitization: All rich text extraction outputs are sanitized, removing unicode emojis and control characters before entering the parsing pipeline.

4.3 External Integration Interfaces

- Supabase Client: Standard JS client listening to real-time replication channels:

```
const channel = supabase
  .channel('resumes_updates')
  .on('postgres_changes', { event: 'UPDATE', schema: 'public', table: 'resumes' },
payload => {
  updateUIKanban(payload.new);
})
  .subscribe();
```

- NVIDIA NIM Endpoints: Secure HTTPS integrations requiring valid OAuth Bearer keys.
- Cloudflare R2 Storage: Secure AWS S3-compatible endpoints utilized strictly for encrypted database snapshots.